

STEP-BY-STEP GUIDE

Integrating Yotsuba Shader into your engine

Everything you need to give the programmers who use your framework typed, validated shaders written in C\#

Edition 1.0 · October 2026 · Yotsuba Shader is released under the Microsoft Public License (MS-PL)

Contents

Before you start **1**
What you will build 1
How to read it 1
1 How Yotsuba Shader fits into an engine **2**
1.1 The pipeline 2
1.2 What Yotsuba Shader does, and what your engine does 2
1.3 The five integration points 2
1.4 Three decisions to make now 2
2 Set up the solution **4**
2.1 The projects 4
2.2 The language package 4
2.3 The compilation step 4
2.4 Requirements 5
3 Your types become GPU types **6**
3.1 How the GPU sees your types 6
3.2 Scalars: nothing to do 6
3.3 Vectors 6
3.4 Matrices 7
3.5 Textures 8
3.6 Samplers 8
3.7 Structs and arrays 9
3.8 A type you do not own 9
3.9 Check that it works 9
4 Write shaders with your types **10**
4.1 Stages 10
4.2 Interface slots 10
4.3 Choose one syntax for your engine 10
4.4 A complete example 11
5 Resources and the binding convention **13**
5.1 Uniforms, storage buffers, textures and samplers 13
5.2 Binding slots 13
5.3 A convention that scales 13
5.4 Uniform data and memory 14
6 Compile your shaders **15**
6.1 The source generator 15
6.2 Two objects on every shader 15
6.3 The formats 16
6.4 From text to something the GPU runs 16
6.5 The command line 16
6.6 Translating at run time 17
7 The CPU side: typed shader parameters **18**
7.1 The problem 18
7.2 Step 1: hide how the graphics API sets a uniform 18
7.3 Step 2: a base class for every shader 18
7.4 Step 3: a shader on top of it 19
7.5 Step 4: logic on the CPU, once per draw 19
7.6 Step 5: fail when the shader loads 20
7.7 When your engine already has a base type 20
8 Load shaders and create pipelines **22**

8.1 One interface for the graphics API 22
8.2 Choose the format for the device 22
8.3 The loader 22
8.4 What each backend does with a module 24
8.5 Uniform buffers and the `std140` layout 24
9 Materials and the render loop 26
9.1 Using a shader in a game 26
9.2 Keep the update frequency visible 26
9.3 A shader object per material 26
9.4 Performance notes 26
9.5 Hot iteration 27
10 Test without a graphics card 28
10.1 The CPU side is plain C# 28
10.2 Test the reflection 28
10.3 Test the layout 29
10.4 Validate the binaries in CI 29
11 Adopt it in an existing engine 30
11.1 Step 1: mark your types 30
11.2 Step 2: write one new shader 30
11.3 Step 3: run both side by side 30
11.4 Step 4: replace string parameters 30
11.5 Step 5: remove the old path 30
11.6 What to measure 30
12 Ship it 31
12.1 Package your engine 31
12.2 Keep these stable 31
12.3 Shader libraries 31
12.4 Licenses 31
12.5 Document it for your users 32
13 Troubleshooting 33
Appendix A: Checklist 34
Appendix B: Built-in values across syntaxes 35
Appendix C: Resource attributes across syntaxes 37
Appendix D: Glossary 38
Appendix E: Frequently asked questions 39

Before you start

This guide is for the person who builds an engine, a framework or a content tool in C# and wants it to run shaders written with Yotsuba Shader. It follows one imaginary engine, `MyEngine`, from an empty solution to a tested and published package. Every step is something you copy into your own project.

What you will build

When you finish, the programmers who use your engine will write this:

Game.cs

```
var lit = new Lit(program.Parameters);

lit.Material = new MaterialData { Roughness = 0.4f };
lit.Albedo = assets.Load<Texture2D>("brick.png");
```

and every part of that line will be checked: the name, the type and the existence of the parameter. To get there you will build seven things:

No.	You build	Chapter
1	The types of your engine, marked so the GPU understands them	3
2	The conventions for slots and update frequency	5
3	The build step that compiles every shader to the formats you need	6
4	A base class that turns uniforms into typed properties	7
5	A loader that creates pipelines from the compiled shader	8
6	Tests that run without a graphics card	10
7	A package that your users install	12

How to read it

- Chapters 1 and 2 give the map and the setup. Read them first.
- Chapters 3 to 5 are about the shader language and the data. You can read them in any order when you need one topic.
- Chapters 6 to 9 are the integration itself. Follow them in order.
- Chapters 10 to 13 are quality, adoption in an existing engine, shipping and troubleshooting.

Every code listing has a file name on top. The listings in this guide are real files in the `code/` folder next to this document, and the C# ones are compiled against the language package every time the guide is built, so you can copy them with confidence. The Spanish edition has the same structure.

> **NOTE**
>
> A listing shows the code of one file. When a file only shows a few members, a comment on the first line tells you which class they belong to.

CHAPTER 1

How Yotsuba Shader fits into an engine

1.1 The pipeline

Yotsuba Shader turns a C# class into shader binaries. Your engine never parses shader text and never ships a second language. The compiler reads the class, checks its interfaces and resources, and produces SPIR-V. The other formats come from SPIR-V.



Two translators sit behind the other formats: SPIRV-Cross writes HLSL and Metal Shading Language, and Naga writes WGSL and GLSL. You do not call them directly. You ask for a format and receive its text.

1.2 What Yotsuba Shader does, and what your engine does

Yotsuba Shader	Your engine
Defines the attributes that describe types, stages, slots and resources	Applies them to its own vector, matrix and texture types
Finds the shaders in your project and checks them while you build	Decides the binding convention and the update frequency of each group of data
Generates every binary you ask for and a description of each shader	Picks the format that matches the graphics API at run time
Exposes the same two objects on every shader: its code and its reflection	Creates pipelines, buffers and descriptors from them
Skips CPU code in the shader class that you mark with an attribute	Gives every shader a typed API on the CPU

The line between the two columns is the subject of this guide. The left column is already done. The right column is what you build.

1.3 The five integration points

Point	Question it answers	Chapter
Types	Which of my types are a vector, a matrix, a texture or a sampler?	3
Language	How do my users describe stages, inputs and outputs?	4
Resources	Where does each uniform, buffer and texture go?	5
Compilation	Which formats do I generate, and when?	6
CPU API	How does the game set a uniform and load a shader?	7 and 8

1.4 Three decisions to make now

Write down your answers before you begin. Changing them later is possible, but it touches every shader your users have written.

1. **Which syntax do you recommend?** The HLSL, GLSL, MSL, SPIR-V and WGSL syntaxes express the same intent. The syntax does not choose the output. Choose the one most of your users already know, and use it in all your examples. See Chapter 4.
2. **Which formats do you ship?** Only the graphics APIs your engine supports. Each format you drop makes the build faster and the package smaller. See Chapter 6.
3. **What is your binding convention?** A rule that says which `set` holds the camera, which one holds the material and which one holds the object. Chapter 5 proposes one.

CHAPTER 2

Set up the solution

2.1 The projects

Split the engine so that the types, the graphics layer and the shaders can change at different speeds.

Solution layout

```
MyEngine/
├── MyEngine.sln
├── src/
│   ├── MyEngine.Core/            vector, matrix and texture types
│   │   ├── Vector2.cs  Vector3.cs  Vector4.cs  Matrix4.cs
│   │   └── Texture2D.cs  SamplerState.cs
│   ├── MyEngine.Graphics/        the graphics API layer
│   │   ├── IGraphicsDevice.cs  IParameterSet.cs
│   │   ├── ShaderProgram.cs  Std140.cs
│   │   └── Vulkan/  Direct3D12/  Metal/  OpenGL/
│   └── MyEngine.Shaders/         shader base class and engine shaders
│       ├── ShaderBase.cs
│       └── Lit.cs  Unlit.cs  Tonemap.cs
├── samples/
│   └── Game/                     a game that uses the engine
└── tests/
    └── MyEngine.Shaders.Tests/
```

Project	References	Why
MyEngine.Core	YotsubaShader.Language	Your types carry the attributes. The package only holds metadata, so it does not change how they behave or how they sit in memory.
MyEngine.Graphics	MyEngine.Core, YotsubaShader.Language	Reads the compiled shaders and talks to the graphics API.
MyEngine.Shaders	Core, Graphics, the source generator	Contains the shaders your engine ships. The generator compiles them during the build.
Game	MyEngine.Shaders	Uses the shaders through their typed API.

2.2 The language package

The package with the attributes goes in the project that defines your types.

MyEngine.Core.csproj

```
<Project Sdk="Microsoft.NET.Sdk">

  <PropertyGroup>
    <TargetFramework>net8.0</TargetFramework>
    <Nullable>enable</Nullable>
  </PropertyGroup>

  <ItemGroup>
    <!-- Only metadata: it does not change your types or their layout -->
    <PackageReference Include="YotsubaShader.Language" Version="1.*" />
  </ItemGroup>

</Project>
```

2.3 The compilation step

The shader project references the source generator and lists the formats you ship in `YotsubaTargets`.

MyEngine.Shaders.csproj

```
<Project Sdk="Microsoft.NET.Sdk">

  <PropertyGroup>
    <TargetFramework>net8.0</TargetFramework>
    <Nullable>enable</Nullable>
  </PropertyGroup>

  <ItemGroup>
    <PackageReference Include="YotsubaShader.SourceGenerator" Version="1.*"
                      PrivateAssets="all" />
    <ProjectReference Include="..\MyEngine.Core\MyEngine.Core.csproj" />
    <ProjectReference Include="..\MyEngine.Graphics\MyEngine.Graphics.csproj" />
  </ItemGroup>

  <PropertyGroup>
    <YotsubaTargets>
      spirv;hlsl;msl;wgsl;glsl;glsles
    </YotsubaTargets>
  </PropertyGroup>

</Project>
```

The generator runs inside `dotnet build`. It needs no extra tool on your machine and no step in your CI beyond the build itself. A shader that does not compile fails the build like any C# error. Chapter 6 explains each target.

> **TIP**
>
> Use `PrivateAssets="all"` on the generator. It keeps the generator out of the package you publish, so your users do not receive a build tool they never asked for.

2.4 Requirements

- The .NET 8 SDK, on macOS, Linux or Windows.
- Nothing else to compile shaders. The graphics SDKs you already use for your backends are still yours to install.

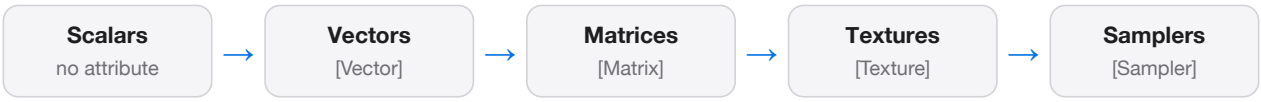
CHAPTER 3

Your types become GPU types

If you build an engine, you already have `Vector2`, `Matrix4` and `Texture2D`. You do not need a second set of types for shaders. You add attributes to the types you already have, and the programmers who use your engine write shaders with the types they use everywhere else.

3.1 How the GPU sees your types

To the GPU, a struct without attributes is only a group of fields. The name does not matter: a struct called `Vector2` is not a vector unless you say so. You mark only the types that must behave as a vector, a matrix, a texture or a sampler.



3.2 Scalars: nothing to do

`bool`, `int`, `uint`, `float` and `double` are GPU types as they are. Use them alone or inside structs.

3.3 Vectors

Put `[Vector]` on the struct and `[VectorComponent(i)]` on each component, starting at 0. The index is the position of the component in the GPU vector, so `X` is 0, `Y` is 1, and so on. The number of members you mark is the size of the vector.

Vector2.cs

```
using YotsubaShader.Language.Native.Types.Vector;

namespace MyEngine;

[Vector]
public struct Vector2
{
    [VectorComponent(0)] public float X;
    [VectorComponent(1)] public float Y;

    public Vector2(float x, float y) { X = x; Y = y; }

    public static Vector2 operator +(Vector2 a, Vector2 b)
    {
        return new Vector2(a.X + b.X, a.Y + b.Y);
    }
}
```

Your constructors, operators and methods stay as they are. Components can be fields or properties, and a vector can have `int` or `uint` components when all of them share one type.

Vector3.cs

```cs
using YotsubaShader.Language.Native.Types.Vector;

namespace MyEngine;

[Vector]
public partial struct Vector3
{
    [VectorComponent(0)] public float X { get; set; }
    [VectorComponent(1)] public float Y { get; set; }
    [VectorComponent(2)] public float Z { get; set; }

    public Vector3(float x, float y, float z) { X = x; Y = y; Z = z; }

    public static Vector3 Normalize(Vector3 v)
    {
        float length = MathF.Sqrt(v.X * v.X + v.Y * v.Y + v.Z * v.Z);
        return new Vector3(v.X / length, v.Y / length, v.Z / length);
    }
}
```

Vector4.cs

```cs
using YotsubaShader.Language.Native.Types.Vector;

namespace MyEngine;

[Vector]
public struct Vector4
{
    [VectorComponent(0)] public float X;
    [VectorComponent(1)] public float Y;
    [VectorComponent(2)] public float Z;
    [VectorComponent(3)] public float W;

    public Vector4(float x, float y, float z, float w)
    {
        X = x; Y = y; Z = z; W = w;
    }
}
```

IntVectors.cs

```cs
// Integer vectors: all the components share one type
[Vector]
public struct Point2
{
    [VectorComponent(0)] public int X;
    [VectorComponent(1)] public int Y;
}

[Vector]
public struct UPoint3
{
    [VectorComponent(0)] public uint X;
    [VectorComponent(1)] public uint Y;
    [VectorComponent(2)] public uint Z;
}
```

CAREFUL

The components of one vector must have the same type, and only `float`, `int` and `uint` are valid. A mixed vector throws `InconsistentVectorComponentTypeException` and a `double` component throws `VectorComponentTypeNotValidException`. Both appear when the compiler processes a shader that uses the type, so compile one shader that uses each of your types before you publish them.

3.4 Matrices

A matrix is a struct marked with `[Matrix]` whose columns are vectors. Mark each column with `[MatrixColumn(i)]`, starting at 0. The column type is one of the vectors you declared.

Matrix4.cs

```
using YotsubaShader.Language.Native.Types.Matrix;

namespace MyEngine;

[Matrix]
public struct Matrix4
{
    [MatrixColumn(0)] public Vector4 Column0;
    [MatrixColumn(1)] public Vector4 Column1;
    [MatrixColumn(2)] public Vector4 Column2;
    [MatrixColumn(3)] public Vector4 Column3;
}
```

3.5 Textures

`[Texture]` goes on a class or a struct. It needs two values: the dimension (`D1`, `D2`, `D3` or `Cube`) and the kind of values it holds (`Float`, `Int` or `Uint`). The optional settings describe the rest.

Textures.cs

```
using YotsubaShader.Language.Native.Types.Texture;

namespace MyEngine;

[Texture(TextureDimension.D2, TextureComponentTypes.Float)]
public class Texture2D { /* your engine's texture */ }

[Texture(TextureDimension.Cube, TextureComponentTypes.Float)]
public class TextureCube { /* ... */ }

[Texture(TextureDimension.D2, TextureComponentTypes.Float,
    Depth = TextureDepth.Yes)]
public class DepthTexture2D { /* ... */ }

[Texture(TextureDimension.D2, TextureComponentTypes.Float,
    Usage = TextureUsage.Storage, Format = TextureFormat.Rgba8)]
public class StorageTexture2D { /* ... */ }
```

Setting	Values	Default
`Usage`	`Sampled` (the shader reads it), `Storage` (the shader writes it)	`Sampled`
`Depth`	`No`, `Yes`, `Unknown`	`No`
`Format`	`Unknown`, `Rgba8`, `Rgba16f`, `Rgba32f`, `R32f`, `R32i`, `R32ui` and others	`Unknown`
`Arrayed`	`true` for an array of textures	`false`
`Multisampled`	`true` for a multisampled texture	`false`

A storage texture also needs its pixel `Format`, because the shader writes to it.

3.6 Samplers

A sampler decides how a texture is read. It has no settings: `[Sampler]` only says that the type represents one.

SamplerState.cs

```
using YotsubaShader.Language.Native.Types;

namespace MyEngine;

[Sampler]
public class SamplerState { /* your engine's sampler */ }
```

3.7 Structs and arrays

Any other struct is converted field by field, including nested structs. Arrays need no attribute. What changes is where the data comes from: `[Uniform]` and `[StorageBuffer]` mark values that your engine uploads before drawing. Chapter 5 covers them.

3.8 A type you do not own

Attributes go on types you control. You cannot mark `System.Numerics.Vector3` or a type from another package. Keep your own struct and convert at the edges of your code. A `partial` struct lets you add the conversions in a separate file.

Interop.cs

```
namespace MyEngine;

// You cannot put attributes on System.Numerics.Vector3: it is not your type.
// Keep your own struct and convert at the edges of your code.
public partial struct Vector3
{
    public static implicit operator Vector3(System.Numerics.Vector3 v)
    {
        return new Vector3(v.X, v.Y, v.Z);
    }

    public static implicit operator System.Numerics.Vector3(Vector3 v)
    {
        return new System.Numerics.Vector3(v.X, v.Y, v.Z);
    }
}
```

3.9 Check that it works

Open the playground or compile a small project, put your types in one file and a shader that uses each of them in another. Two things should happen: there are no errors, and the entry points of the shader are listed. Do this once for every type you publish. It is the cheapest test you will write.

CHAPTER 4

Write shaders with your types

4.1 Stages

A shader is a class. Its entry points are methods marked with `[ShaderStage]`. A class can have any number of stages, and methods without the attribute are ordinary C# that the compiler ignores.

Stage	Receives	Returns
`Stage.Vertex`	Per-vertex input	A struct that includes the clip-space position
`Stage.Fragment`	The interpolated values of the vertex stage	The color for the render target
`Stage.Compute`	A built-in such as the global invocation id	Nothing: it writes to storage buffers

4.2 Interface slots

Every value that crosses a stage boundary has a slot. There are two kinds you write: numbered locations and built-in values.

Types.cs

```
using MyEngine;
using YotsubaShader.Language;
using YotsubaShader.HlslSyntax;

public struct VertexInput
{
    [Location(0)] public Vector3 Position;
    [Location(1)] public Vector3 Normal;
    [Location(2)] public Vector2 UV;
}

public struct VertexOutput
{
    [SV_Position] public Vector4 Position;
    [Location(0)] public Vector3 Normal;
    [Location(1)] public Vector2 UV;
}
```

`[Location(n)]` links an output of one stage to the input of the next with the same number. `[SV_Position]` is a built-in: the clip-space position that the GPU needs. Each syntax names the built-ins in its own way. Appendix B lists them side by side.

4.3 Choose one syntax for your engine

The five syntaxes are equivalent. Each one is its own package, with a namespace of the same name, built on `YotsubaShader.Language`. The HLSL syntax does not require a Direct3D target, and the WGSL syntax does not require WebGPU. Choose by the habits of your users, not by the graphics API you support.

If your users…	Recommend
come from Unity, Direct3D or XNA and MonoGame	HLSL: `YotsubaShader.HlslSyntax`
come from OpenGL or Godot	GLSL: `YotsubaShader.GlslSyntax`
write for Apple platforms	MSL: `YotsubaShader.MslSyntax`
work close to Vulkan and SPIR-V	SPIR-V: `YotsubaShader.SpirvSyntax`
write for the web	WGSL: `YotsubaShader.WgslSyntax`
have no habit yet	HLSL: the link between stages is easy to see

> **CAREFUL**
>
> Several names exist in more than one syntax (`uniform` and `buffer` , for example). Import one syntax namespace per file, and tell your users to do the same.

4.4 A complete example

The shader below is the one you will carry through the rest of the guide. It uses the types of the previous chapter, three uniform groups and a texture with its sampler.

UniformData.cs

```
using MyEngine;

public struct Light
{
    public Vector3 Direction;
    public float Intensity;
}

public struct FrameData
{
    public Matrix4 ViewProjection;
    public Vector3 CameraPosition;
    public float Time;
    public Light Sun;
}

public struct MaterialData
{
    public Vector4 Tint;
    public float Roughness;
}

public struct ObjectData
{
    public Matrix4 World;
}
```

Lit.cs

```
using MyEngine;
using MyEngine.Graphics;
using MyEngine.Shaders;
using YotsubaShader.Language;
using YotsubaShader.Language.Resources;

public partial class Lit : ShaderBase
{
    public Lit(IParameterSet parameters) : base(parameters, Reflection) { }

    // Uniforms: one line each. The name comes from the property.
    [Uniform, Binding(Sets.Frame, 0)]
    public FrameData Frame { get => Get<FrameData>(); set => Set(value); }

    [Uniform, Binding(Sets.Material, 0)]
    public MaterialData Material { get => Get<MaterialData>(); set => Set(value); }

    [Uniform, Binding(Sets.Object, 0)]
    public ObjectData Object { get => Get<ObjectData>(); set => Set(value); }

    // Textures and samplers
    [Binding(Sets.Material, 1)]
    public Texture2D Albedo { get => GetTexture(); set => SetTexture(value); }

    [Binding(Sets.Material, 2)]
    public SamplerState Sampler { get => GetSampler(); set => SetSampler(value); }

    [ShaderStage(Stage.Vertex)]
    public VertexOutput Vertex(VertexInput input)
    {
        VertexOutput output;
        output.Position = new Vector4(
            input.Position.X, input.Position.Y, input.Position.Z, 1);
        output.Normal = input.Normal;
        output.UV = input.UV;
        return output;
    }

    [ShaderStage(Stage.Fragment)]
```

```
    public Vector4 Fragment(VertexOutput input)
    {
        return Material.Tint;
    }
}
```

You have not seen `ShaderBase`, `Get` and `Set` yet. They are the CPU side of the shader and Chapter 7 builds them. For now, read the stage methods and the attributes on the properties.

CHAPTER 5

Resources and the binding convention

5.1 Uniforms, storage buffers, textures and samplers

Attribute	What it is for	Typical size
`[Uniform]`	A value that stays the same during a draw or dispatch: a camera, a time, a material	Small
`[StorageBuffer]`	An array the shader can read and write: particles, results of a computation	Large
A texture field	The pixels of an image, with `[Texture]` on its type	Any
A sampler field	How a texture is read, with `[Sampler]` on its type	—

A storage buffer is read and write by default. State the access when the shader only reads or only writes: `Access = ResourceAccess.ReadOnly` or `WriteOnly`. A type can also say it for you. `ImmutableArray<T>`, `ReadOnlyMemory<T>` and `ReadOnlySpan<T>` are read-only because of what they are. In a class field use `ImmutableArray<T>` or `ReadOnlyMemory<T>`, because C# does not allow spans as fields.

> **CAREFUL**
>
> A field cannot be both `[Uniform]` and `[StorageBuffer]`. The compiler throws `InvalidOperationException`. Choose a uniform for a small value that does not change during the draw, and a storage buffer for an array.

5.2 Binding slots

Your engine needs to know where each resource goes. The shared `[Binding(set, binding)]` attribute works in every syntax, and each syntax also has the spelling of its source language.

The same slot in each syntax

```
[Uniform, Binding(1, 0)]                  // any syntax
[cbuffer, register(RegisterPrefix.b, 0)]   // HLSL
[uniform, layout(1, 0)]                   // GLSL
[constant, buffer(0)]                     // MSL
[Uniform, DescriptorSet(1), @Binding(0)]  // SPIR-V
[uniform, @group(1), @binding(0)]         // WGSL
```

The slots are only numbers. The meaning of each number is your convention.

5.3 A convention that scales

A good convention makes the update frequency visible. Put what changes less in a lower set. The engine then binds each set once per group of draws, not once per draw.

Sets.cs

```
namespace MyEngine.Shaders;

// One rule for the whole engine: what changes less lives in a lower set.
public static class Sets
{
    public const int Frame = 0;      // once per frame: camera, time, lights
    public const int Material = 1;   // once per material: textures, factors
    public const int Object = 2;     // once per object: world matrix
}
```

Set	Holds	Changes	Bound
0 · Frame	Camera, time, lights	Once per frame	Once per frame
1 · Material	Textures, samplers, material factors	Once per material	When the material changes
2 · Object	World matrix, per-object data	Once per object	Once per draw

Tell your users to use these constants and never numbers. A shader written with `Sets.Material` keeps working if you later reorder the sets.

5.4 Uniform data and memory

A uniform struct is uploaded as plain memory. The rules for how the fields sit in that memory come from the graphics API, not from Yotsuba Shader. The attributes do not change the layout of your types. Chapter 8 shows a small helper that computes the standard `std140` layout from the same attributes you added in Chapter 3, so your uploader and the shader agree.

CHAPTER 6

Compile your shaders

6.1 The source generator

This is the main way to compile. A Roslyn source generator runs as part of `dotnet build`. It finds every `[ShaderStage]` method, compiles it and writes the result into the other half of the class. For this the class must be `partial`.

Triangle.cs

```
public partial class Triangle
{
    [ShaderStage(Stage.Vertex)]
    public VertexOutput Vertex([SV_VertexID] uint id) { /* ... */ }

    [ShaderStage(Stage.Fragment)]
    public Vector4 Fragment(VertexOutput input) { /* ... */ }
}
```

The generator adds this half. It declares two fields and fills them with generated code.

Triangle.g.cs

```
// <auto-generated by Yotsuba Shader />
using YotsubaShader.Language.Compilation;

public partial class Triangle
{
    [ShaderIgnore]
    public static readonly ShaderCompiledCode CompiledCode =
        /* ... Autogenerated code with all the shader binaries
           that you specified */;

    [ShaderIgnore]
    public static readonly ShaderReflection Reflection =
        /* ... Autogenerated code with the entry points and
           resources of this shader */;
}
```

Both fields carry `[ShaderIgnore]`, which tells Yotsuba Shader not to treat them as part of the shader. The generated half adds no base class, so your own base class, such as the `ShaderBase` of Chapter 7, never conflicts with it.

6.2 Two objects on every shader

Every shader exposes the same two types. That is what lets your engine load any shader in the same way.

YotsubaShader.Language.Compilation

```
public sealed class ShaderCompiledCode
{
    public bool Has(string entryPoint, Binaries format);
    public ReadOnlySpan<byte> Get(string entryPoint, Binaries format);
    public string GetText(string entryPoint, Binaries format);
}

public sealed class ShaderReflection
{
    public IReadOnlyList<Binaries> BinaryFormats { get; }
    public IReadOnlyList<EntryPointInfo> EntryPoints { get; }
    public IReadOnlyList<ResourceBinding> Resources { get; }
}
```

- `ShaderCompiledCode` holds one binary per entry point and format. The key is the name of the C# method, so `nameof(Triangle.Vertex)` is checked by the compiler. Text formats come back as UTF-8 bytes, and `GetText` decodes them.

- `ShaderReflection` lists the formats that were generated, the entry points (name and stage) and the resources (name, kind, access and slot). You never read a binary to build a pipeline.

ReadBinaries.cs

```
using YotsubaShader.Language.Compiler;

// One entry point, one format
var vertex = Triangle.CompiledCode.Get(
    nameof(Triangle.Vertex), Binaries.SpirV);

// Any shader, the same way
foreach (var entry in Triangle.Reflection.EntryPoints)
{
    var bytes = Triangle.CompiledCode.Get(entry.Name, Binaries.SpirV);

    // ... your graphics and compute pipeline implementation
}
```

The data lives as static data in your assembly, so reading it neither copies nor allocates.

6.3 The formats

Target	`Binaries`	What it holds	Used for
`spirv`	`Binaries.SpirV`	SPIR-V binary	Vulkan
`hlsl`	`Binaries.Hlsl`	UTF-8 text	Direct3D 11 and 12
`msl`	`Binaries.Msl`	UTF-8 text	Metal
`wgsl`	`Binaries.Wgsl`	UTF-8 text	WebGPU
`glsl`	`Binaries.Glsl`	UTF-8 text	OpenGL
`glsles`	`Binaries.GlslEs`	UTF-8 text	OpenGL ES, WebGL

Only the targets listed in `YotsubaTargets` are generated. SPIR-V is the format the compiler writes. HLSL and MSL are translated from it by SPIRV-Cross, and WGSL and GLSL by Naga.

6.4 From text to something the GPU runs

The text formats still need a last step, and it is the job of the platform:

API	Who compiles the text	You ship
Vulkan	Nobody: SPIR-V is the final binary	SPIR-V
Metal	The Metal framework, when you create the library	MSL text
WebGPU	The browser	WGSL text
OpenGL, WebGL	The graphics driver	GLSL text
Direct3D 11 and 12	DXC (Shader Model 6) or FXC (Shader Model 5.x), which you already use	HLSL text

Yotsuba Shader stops at the HLSL text. It does not produce DXIL or DXBC. If you want Direct3D bytecode inside your package, compile the HLSL with your own DXC or FXC in the same build step and ship the result.

6.5 The command line

For CI, scripts and content tools, the same compiler is available as a command. You give it your project file and it writes one binary per entry point.

Terminal

```
yotsuba compile YourGame.csproj \
  --target spirv \
  --output artifacts/shaders
```

The output folder holds `Triangle.Vertex.spv`, `Triangle.Fragment.spv` and so on. Add `--entry Triangle.Vertex` to compile one entry point. This is also the easiest way to inspect the output with `spirv-val` and `spirv-dis`.

Workflow	Runs	Best for	Output
Source generator	During `dotnet build`	Engines and games that ship shaders with the code	Static data in a `partial` class
Command line	In scripts and CI	Content pipelines, inspection, validation	`.spv` files

6.6 Translating at run time

Most engines never need this: the generator already produced every format. When your tool creates shaders while it runs, such as an editor, hand the SPIR-V to the `Transpile` API and receive text.

Transpile.cs

```
using YotsubaShader.Transpile;

var hlslOptions = new HlslOptions { ShaderModel = 51, EntryPoint = "Vertex" };
var glslOptions = new GlslOptions { Es = true, Version = 300 };

string hlsl = Transpile.FromSpirVToHlsl(spirv, hlslOptions);
string msl  = Transpile.FromSpirVToMsl(spirv);
string wgsl = Transpile.FromSpirVToWgsl(spirv);
string glsl = Transpile.FromSpirVToGlsl(spirv, glslOptions);
```

Both translators are native code, so each platform needs its own build. On consoles, translate at build time: neither translator was verified against any console toolchain.

CHAPTER 7

The CPU side: typed shader parameters

7.1 The problem

Every engine sets a uniform by name, usually with a string:

The usual way

```
parameters.Set("Exposure", 1.2f);
parameters.Set("LightDirection", direction);
```

A typo, a parameter renamed in the shader but not in the game, or a value of the wrong type all compile without errors. They fail when that line runs, not when you build. Yotsuba Shader removes the problem because the shader class is C#: the name and the type of each uniform are written once, and the C# compiler checks them when the shader is built and when the game uses it.

7.2 Step 1: hide how the graphics API sets a uniform

Vulkan, Direct3D, Metal and OpenGL set uniforms in different ways. Put that difference behind one interface. It is the only part of this chapter that depends on the graphics API.

IParameterSet.cs

```
namespace MyEngine.Graphics;

public interface IParameterSet
{
    bool Has(string name);

    // Plain data: numbers, vectors, matrices and structs of them
    T Get<T>(string name) where T : unmanaged;
    void Set<T>(string name, in T value) where T : unmanaged;

    // Resources: they are objects of your engine, not plain data
    Texture2D? GetTexture(string name);
    void SetTexture(string name, Texture2D texture);

    SamplerState? GetSampler(string name);
    void SetSampler(string name, SamplerState sampler);
}
```

Each backend implements it once. A Vulkan version writes into a uniform buffer, and an OpenGL version calls the uniform functions. Plain data goes through `Get` and `Set`, so `T` is `unmanaged`. Textures and samplers are objects of your engine and have their own methods.

7.3 Step 2: a base class for every shader

The base class holds the parameter set and gives each shader the helpers it needs. The name of the uniform is not a parameter that the programmer writes: `[CallerMemberName]` makes the C# compiler fill it with the name of the property that calls the helper.

ShaderBase.cs

```
using System.Runtime.CompilerServices;
using MyEngine.Graphics;
using YotsubaShader.Language.Compiler;
using YotsubaShader.Language.Compilation;

namespace MyEngine.Shaders;

[ShaderIgnore]
public abstract class ShaderBase
{
    [ShaderIgnore]
    private readonly IParameterSet parameters;
```

```
    [ShaderIgnore]
    protected ShaderBase(IParameterSet parameters, ShaderReflection reflection)
    {
        this.parameters = parameters;

        // Fail when the shader loads, not on the first frame that draws with it
        foreach (var resource in reflection.Resources)
        {
            if (!parameters.Has(resource.Name))
            {
                throw new InvalidOperationException(
                    $"The shader has no parameter named '{resource.Name}'.");
            }
        }
    }

    [ShaderIgnore]
    protected T Get<T>([CallerMemberName] string name = "") where T : unmanaged
        => parameters.Get<T>(name);

    [ShaderIgnore]
    protected void Set<T>(T value, [CallerMemberName] string name = "") where T : unmanaged
        => parameters.Set(name, value);

    [ShaderIgnore]
    protected Texture2D GetTexture([CallerMemberName] string name = "")
        => parameters.GetTexture(name)!;

    [ShaderIgnore]
    protected void SetTexture(Texture2D texture, [CallerMemberName] string name = "")
        => parameters.SetTexture(name, texture);

    [ShaderIgnore]
    protected SamplerState GetSampler([CallerMemberName] string name = "")
        => parameters.GetSampler(name)!;

    [ShaderIgnore]
    protected void SetSampler(SamplerState sampler, [CallerMemberName] string name = "")
        => parameters.SetSampler(name, sampler);
}
```

Inside `public MaterialData Material { get => Get<MaterialData>(); }` the name is `"Material"`. There is no string and no `nameof` to forget, and renaming the property renames the uniform. Everything in this class is CPU code, so it carries `[ShaderIgnore]`.

7.4 Step 3: a shader on top of it

The shader of Chapter 4 already has this shape. Look at its properties again:

Lit.cs (excerpt)

```
[Uniform, Binding(Sets.Material, 0)]
public MaterialData Material { get => Get<MaterialData>(); set => Set(value); }

[Binding(Sets.Material, 1)]
public Texture2D Albedo { get => GetTexture(); set => SetTexture(value); }
```

One line per resource. The constructor passes the reflection of the shader to the base class: `base(parameters, Reflection)`. `Reflection` is the static field that the generated half declares.

7.5 Step 4: logic on the CPU, once per draw

A shader runs its code for every vertex and every pixel. Anything that is the same for the whole draw is cheaper on the CPU. Add it to the same class and mark it `[ShaderIgnore]`.

LitLogic.cs

```
// Inside the Lit class

// CPU only: the compiler skips it
[ShaderIgnore]
public float Brightness
{
    get => Material.Tint.W;
    set
    {
        var material = Material;
        material.Tint.W = value;
        Material = material;
    }
}

[ShaderIgnore]
public void Fade(float factor) => Brightness *= factor;
```

The game never has to know how the tint is stored.

7.6 Step 5: fail when the shader loads

The constructor of `ShaderBase` compares `reflection.Resources` with the parameter set. A resource that the compiled shader has and the engine cannot provide throws when the shader loads, with the name of the resource in the message. Without this check the same mistake would appear as a black screen on the first frame that draws.

7.7 When your engine already has a base type

Some engines give you a class to inherit from. The idea does not change: the property calls the engine, and the rest of the game uses the property.

Engine	How the class reaches it	Listing
XNA, MonoGame, FNA	Inherit from `Microsoft.Xna.Framework.Graphics.Effect` and use `Parameters`	LitXna.cs
Unity	Hold the `Material` in a field marked `[ShaderIgnore]`	LitUnity.cs
Godot	Inherit from `ShaderMaterial` and use `SetShaderParameter`	LitGodot.cs

LitXna.cs

```
using Microsoft.Xna.Framework;
using Microsoft.Xna.Framework.Graphics;
using YotsubaShader.Language.Resources;

public partial class Lit : Effect
{
    // Wraps the effect that your game loaded
    public Lit(Effect cloneSource) : base(cloneSource) { }

    [Uniform]
    public float Exposure
    {
        get => Parameters[nameof(Exposure)].GetValueSingle();
        set => Parameters[nameof(Exposure)].SetValue(value);
    }

    [Uniform]
    public Vector3 LightDirection
    {
        get => Parameters[nameof(LightDirection)].GetValueVector3();
        set => Parameters[nameof(LightDirection)].SetValue(value);
    }
}
```

LitUnity.cs

```cs
using UnityEngine;
using YotsubaShader.Language.Compiler;
using YotsubaShader.Language.Resources;

public partial class Lit
{
    // CPU only: it is not part of the shader
    [ShaderIgnore]
    private readonly Material material;

    public Lit(Material material) { this.material = material; }

    [Uniform]
    public float Exposure
    {
        get => material.GetFloat(nameof(Exposure));
        set => material.SetFloat(nameof(Exposure), value);
    }

    [Uniform]
    public Vector3 LightDirection
    {
        get => material.GetVector(nameof(LightDirection));
        set => material.SetVector(nameof(LightDirection), value);
    }
}
```

LitGodot.cs

```cs
using Godot;
using YotsubaShader.Language.Resources;

public partial class Lit : ShaderMaterial
{
    [Uniform]
    public float Exposure
    {
        get => GetShaderParameter(nameof(Exposure)).AsSingle();
        set => SetShaderParameter(nameof(Exposure), value);
    }

    [Uniform]
    public Vector3 LightDirection
    {
        get => GetShaderParameter(nameof(LightDirection)).AsVector3();
        set => SetShaderParameter(nameof(LightDirection), value);
    }
}
```

A field that holds an engine object is not a GPU type, so it carries `[ShaderIgnore]`. Inheriting from an engine type does not conflict with the generated half, because that half adds no base class.

CHAPTER 8

Load shaders and create pipelines

8.1 One interface for the graphics API

The loader needs four things from your graphics layer. Keep the interface small. Each backend implements it once.

IGraphicsDevice.cs

```
using YotsubaShader.Language;
using YotsubaShader.Language.Compilation;

namespace MyEngine.Graphics;

public interface IShaderModule : IDisposable { }

public interface IPipelineLayout : IDisposable { }

public interface IGraphicsDevice
{
    GraphicsApi Api { get; }

    IShaderModule CreateShaderModule(
        Stage stage, ReadOnlySpan<byte> code, string entryPoint);

    // One description of every resource: the API builds its own layout
    IPipelineLayout CreatePipelineLayout(
        IReadOnlyList<ResourceBinding> resources);

    IParameterSet CreateParameterSet(IPipelineLayout layout);
}
```

Note that `CreatePipelineLayout` receives the resources straight from the reflection. Your backend does not know about C# shaders: it only receives a list of names, kinds, accesses and slots, and builds its own descriptor sets, root signature or bind group layout.

8.2 Choose the format for the device

One function maps the graphics API to the format that your engine ships for it.

ShaderFormats.cs

```
using YotsubaShader.Language.Compiler;

namespace MyEngine.Graphics;

public enum GraphicsApi
{
    Vulkan, Direct3D11, Direct3D12, Metal, WebGpu, OpenGl, WebGl,
}

public static class ShaderFormats
{
    public static Binaries For(GraphicsApi api) => api switch
    {
        GraphicsApi.Vulkan => Binaries.SpirV,
        GraphicsApi.Direct3D11 or GraphicsApi.Direct3D12 => Binaries.Hlsl,
        GraphicsApi.Metal => Binaries.Msl,
        GraphicsApi.WebGpu => Binaries.Wgsl,
        GraphicsApi.OpenGl => Binaries.Glsl,
        GraphicsApi.WebGl => Binaries.GlslEs,
        _ => throw new ArgumentOutOfRangeException(nameof(api)),
    };
}
```

8.3 The loader

`ShaderProgram.Load<TShader>` reads the two static fields of the generated half, chooses the format, creates one module per entry point and builds the layout from the reflection. It caches the result per device and shader.

ShaderProgram.cs

```
using YotsubaShader.Language.Compilation;

namespace MyEngine.Graphics;

public sealed class ShaderProgram : IDisposable
{
    private static readonly Dictionary<(IGraphicsDevice, Type), ShaderProgram> cache = new();

    private readonly Dictionary<string, IShaderModule> modules;
    private readonly IPipelineLayout layout;

    public IParameterSet Parameters { get; }

    private ShaderProgram(
        Dictionary<string, IShaderModule> modules,
        IPipelineLayout layout,
        IParameterSet parameters)
    {
        this.modules = modules;
        this.layout = layout;
        Parameters = parameters;
    }

    public IShaderModule this[string entryPoint] => modules[entryPoint];

    // ShaderProgram.Load<Lit>(device): the generated half of Lit
    // declares two static fields, CompiledCode and Reflection.
    public static ShaderProgram Load<TShader>(IGraphicsDevice device)
    {
        var key = (device, typeof(TShader));
        if (cache.TryGetValue(key, out var loaded)) return loaded;

        var code = Read<ShaderCompiledCode>("CompiledCode");
        var reflection = Read<ShaderReflection>("Reflection");

        var format = ShaderFormats.For(device.Api);
        if (!reflection.BinaryFormats.Contains(format))
        {
            throw new NotSupportedException(
                $"{typeof(TShader).Name} was not compiled for {device.Api}: " +
                $"add '{format}' to YotsubaTargets.");
        }

        var modules = new Dictionary<string, IShaderModule>();
        foreach (var entry in reflection.EntryPoints)
        {
            modules[entry.Name] = device.CreateShaderModule(
                entry.Stage, code.Get(entry.Name, format), entry.Name);
        }

        var layout = device.CreatePipelineLayout(reflection.Resources);
        var program = new ShaderProgram(modules, layout, device.CreateParameterSet(layout));
        cache[key] = program;
        return program;

        static T Read<T>(string field) =>
            (T)typeof(TShader).GetField(field)!.GetValue(null)!;
    }

    public void Dispose()
    {
        foreach (var module in modules.Values) module.Dispose();
        layout.Dispose();
    }
}
```

NOTE

The loader reads the static fields by name. It needs no registry and no interface on the shader. If your engine is compiled ahead of time and trims unused members, keep `CompiledCode` and `Reflection` with the usual trimming annotations.

If the format is missing, the message tells your user what to change: add it to `YotsubaTargets`. It is a build setting, not a code change.

8.4 What each backend does with a module

API	Input	Creates
Vulkan	SPIR-V bytes	A shader module
Direct3D 11 and 12	HLSL text, compiled by DXC or FXC	A shader object or a pipeline state stage
Metal	MSL text	A library, then a function with the entry point name
WebGPU	WGSL text	A shader module
OpenGL, WebGL	GLSL text	A shader object, then a program

8.5 Uniform buffers and the `std140` layout

The shader reads a uniform struct from memory with a layout that depends on the graphics API. The common choice for uniform buffers is `std140`. Its rules are small: a scalar takes 4 bytes. A two-component vector takes 8 bytes and aligns to 8. A three- or four-component vector aligns to 16, so a three-component vector takes 12 bytes and the next field may fill the gap. A matrix is stored as columns of four components. A struct aligns to 16 and its size rounds up to 16.

You do not need to type those numbers anywhere. The helper below reads the same attributes you put on your types in Chapter 3.

Std140.cs

```
using System.Reflection;
using YotsubaShader.Language.Native.Types.Matrix;
using YotsubaShader.Language.Native.Types.Vector;

namespace MyEngine.Graphics;

// Size and alignment of a type in a uniform buffer (std140 rules).
public static class Std140
{
    public static int AlignUp(int value, int alignment)
        => (value + alignment - 1) / alignment * alignment;

    public static (int Size, int Alignment) Measure(Type type)
    {
        if (type == typeof(float) || type == typeof(int) || type == typeof(uint))
            return (4, 4);

        if (type.IsDefined(typeof(VectorAttribute), false))
        {
            int n = Count(type, typeof(VectorComponentAttribute));
            return (n * 4, n == 2 ? 8 : 16);     // a vec3 has size 12, alignment 16
        }

        if (type.IsDefined(typeof(MatrixAttribute), false))
        {
            int columns = Count(type, typeof(MatrixColumnAttribute));
            return (columns * 16, 16);           // every column takes a full vec4
        }

        // A struct: fields in order, each one at its own alignment
        int offset = 0;
        foreach (var field in type.GetFields(BindingFlags.Public | BindingFlags.Instance))
        {
            var (size, alignment) = Measure(field.FieldType);
            offset = AlignUp(offset, alignment) + size;
        }
        return (AlignUp(offset, 16), 16);        // a struct rounds up to 16
    }

    public static int OffsetOf(Type structType, string fieldName)
    {
        int offset = 0;
        foreach (var field in structType.GetFields(BindingFlags.Public | BindingFlags.Instance))
        {
            var (size, alignment) = Measure(field.FieldType);
            offset = AlignUp(offset, alignment);
            if (field.Name == fieldName) return offset;
            offset += size;
        }
        throw new ArgumentException($"No field named '{fieldName}'.");
```

```
    }

    private static int Count(Type type, Type attribute)
        => type.GetMembers(BindingFlags.Public | BindingFlags.Instance)
               .Count(m => m.IsDefined(attribute, false));
}
```

Std140Test.cs

```
public struct Particle
{
    public Vector3 Position;
    public float Size;
}

public class Std140Tests
{
    [Fact]
    public void A_vec3_takes_12_bytes_and_the_next_float_fills_the_gap()
    {
        Assert.Equal(0, Std140.OffsetOf(typeof(Particle), nameof(Particle.Position)));
        Assert.Equal(12, Std140.OffsetOf(typeof(Particle), nameof(Particle.Size)));
        Assert.Equal(16, Std140.Measure(typeof(Particle)).Size);
    }

    [Fact]
    public void A_matrix_takes_four_full_columns()
    {
        Assert.Equal(64, Std140.Measure(typeof(Matrix4)).Size);
    }
}
```

CAREFUL

Use `uint` or `int` for flags in uniform structs, not `bool`. A `bool` takes one byte on the CPU and four on the GPU.

CHAPTER 9

Materials and the render loop

9.1 Using a shader in a game

This is the whole loading and update code a game needs. It has no strings and no casts.

Game.cs

```
var device = window.CreateDevice(GraphicsApi.Vulkan);

var program = ShaderProgram.Load<Lit>(device);
var lit = new Lit(program.Parameters);

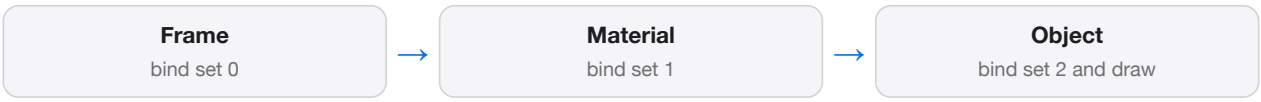
lit.Frame = new FrameData { ViewProjection = camera.ViewProjection, Time = time };
lit.Material = new MaterialData { Tint = new Vector4(1, 1, 1, 1), Roughness = 0.4f };
lit.Albedo = assets.Load<Texture2D>("brick.png");

lit.Material = "rough";   // error CS0029: Cannot implicitly convert type 'string' to 'MaterialData'
```

The last line shows the point of the work: a wrong type is a build error in the game, not a failure during a frame.

9.2 Keep the update frequency visible

The convention of Chapter 5 turns into the render loop:



1. Once per frame, set `lit.Frame` and bind set 0.
2. When the material changes, set its properties and bind set 1. Sort draws by material so this happens as little as possible.
3. For each object, set `lit.Object` and draw.

Because each group is one struct, each group is one upload. Do not set the fields of a uniform struct one at a time: build the struct and assign it.

9.3 A shader object per material

`ShaderProgram` is shared: it holds the modules and the layout. A `Lit` object holds the values of one material. Create one `Lit` per material and one `ShaderProgram` per shader and device. Dispose the programs when the device goes away.

9.4 Performance notes

- **Resolve names once.** The `[CallerMemberName]` string is a constant that the C# compiler writes into your code, so the name costs nothing at run time. Make your `IParameterSet` resolve each name to an offset or a slot when it is created, from the reflection, so every later `Set` is a write into an array and not a lookup by text.
- **No boxing.** `Get` and `Set` are generic and `unmanaged`, so a `float` or a struct is never boxed.
- **One upload per group.** A uniform struct is one write. Assign the whole struct and let the backend upload it once per set.
- **Create material objects when you load, not when you draw.** A `Lit` is cheap, but a loop that creates one per object per frame creates garbage for nothing.

9.5 Hot iteration

During development, the build is your reload. Change the shader, run `dotnet build`, and the generator rewrites the binaries. If your editor must compile shaders while it runs, use the translation API of Chapter 6 on the SPIR-V that the compiler produces.

CHAPTER 10

Test without a graphics card

10.1 The CPU side is plain C#

Because the shader talks to the engine through `IParameterSet`, a test can use a fake one. The logic of your shader classes runs on the build server, where there is no graphics card.

FakeParameterSet.cs

```
public sealed class FakeParameterSet : IParameterSet
{
    private readonly Dictionary<string, object> values = new();

    public bool Has(string name) => true;

    public T Get<T>(string name) where T : unmanaged
        => values.TryGetValue(name, out var value) ? (T)value : default;

    public void Set<T>(string name, in T value) where T : unmanaged
        => values[name] = value;

    public Texture2D? GetTexture(string name)
        => values.TryGetValue(name, out var value) ? (Texture2D)value : null;

    public void SetTexture(string name, Texture2D texture) => values[name] = texture;

    public SamplerState? GetSampler(string name)
        => values.TryGetValue(name, out var value) ? (SamplerState)value : null;

    public void SetSampler(string name, SamplerState sampler) => values[name] = sampler;
}
```

LitTests.cs

```
public class LitTests
{
    [Fact]
    public void Fade_halves_the_alpha_of_the_tint()
    {
        var lit = new Lit(new FakeParameterSet());
        lit.Brightness = 1;

        lit.Fade(0.5f);

        Assert.Equal(0.5f, lit.Brightness);
    }

    [Fact]
    public void The_material_is_stored_under_its_property_name()
    {
        var parameters = new FakeParameterSet();
        var lit = new Lit(parameters);

        lit.Material = new MaterialData { Roughness = 0.4f };

        Assert.Equal(0.4f, parameters.Get<MaterialData>("Material").Roughness);
    }
}
```

10.2 Test the reflection

Every shader exposes its resources. A test that lists the names you expect catches a renamed uniform before your users do.

ReflectionTests.cs

```
public class ReflectionTests
{
    [Fact]
    public void Lit_exposes_the_resources_the_engine_expects()
    {
        var names = Lit.Reflection.Resources.Select(r => r.Name).ToHashSet();

        Assert.Contains("Frame", names);
        Assert.Contains("Material", names);
        Assert.Contains("Albedo", names);
    }
}
```

10.3 Test the layout

The `Std140` helper is plain code, so the tests of Chapter 8 run anywhere. Add a test for every uniform struct of every shader you ship: it is the difference between a correct image and a quiet misalignment.

10.4 Validate the binaries in CI

The build already fails when a shader does not compile. Add one more step: validate the SPIR-V with `spirv-val` from SPIRV-Tools.

.github/workflows/build.yml

```
name: build
on: [push, pull_request]

jobs:
  shaders:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
      - uses: actions/setup-dotnet@v4
        with:
          dotnet-version: 8.0.x

      # The source generator compiles every shader during the build
      - run: dotnet build -c Release

      # The unit tests need no GPU
      - run: dotnet test -c Release --no-build

      # Check the SPIR-V that the build produced
      - run: |
          sudo apt-get update && sudo apt-get install -y spirv-tools
          yotsuba compile src/MyEngine.Shaders/MyEngine.Shaders.csproj \
            --target spirv --output artifacts/shaders
          for f in artifacts/shaders/*.spv; do spirv-val "$f"; done
```

CHAPTER 11

Adopt it in an existing engine

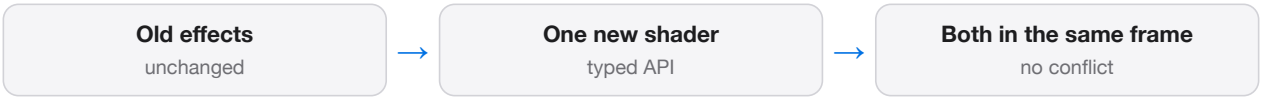
Most engines already have effects, materials and string parameters. You do not have to replace them in one step. This order keeps the engine working after every step.

11.1 Step 1: mark your types

Add the attributes of Chapter 3 to the vector, matrix, texture and sampler types you already have. The attributes are metadata: nothing changes at run time, so this step is safe to release on its own.

11.2 Step 2: write one new shader

Choose a small, self-contained shader, such as a post-processing pass or a debug material. Write it with Yotsuba Shader, load it with the loader of Chapter 8 and draw with it. Keep every other effect as it is.



11.3 Step 3: run both side by side

The new loader and your old one can coexist, because both end in the same graphics API. If you already have a material system, give it a second kind of material that holds a `ShaderBase` object instead of a name and a dictionary of values. Draw both kinds in the same render loop.

11.4 Step 4: replace string parameters

Search your game code for the calls that set a parameter by name. Each one is a candidate for a property:

Before and after

```
// Before: a string, a value of any type
parameters.Set("Exposure", 1.2f);

// After: a property, checked by the compiler
lit.Exposure = 1.2f;
```

Replace them shader by shader. After each shader, build: a name or a type that does not match is now a compiler error, which is the list of what is left to fix.

11.5 Step 5: remove the old path

When no effect uses the old format, remove the old loader and the old parameter code. If the old effect files were text, delete them: the shaders are now C# files in the project, with the rest of your code.

11.6 What to measure

- **Build time.** The generator compiles shaders during the build. Keep `YotsubaTargets` to the formats you ship and compare the time before and after.
- **Package size.** Each format adds its binaries to the assembly. Drop the formats you do not use.
- **Frame time.** It should not change: the GPU receives the same kind of shader as before.

CHAPTER 12

Ship it

12.1 Package your engine

Your users reference one package. It contains your types, which carry the attributes, so a reference to `YotsubaShader.Language` travels with it.

MyEngine.csproj

```
<PropertyGroup>
  <PackageId>MyEngine</PackageId>
  <Version>1.2.0</Version>
  <PackageLicenseExpression>MIT</PackageLicenseExpression>
</PropertyGroup>

<ItemGroup>
  <!-- Your users get the attributes together with your types -->
  <PackageReference Include="YotsubaShader.Language" Version="1.*" />
</ItemGroup>
```

Ship the shaders as part of the package. The binaries are static data inside your assembly, so there are no extra files to copy.

12.2 Keep these stable

Your users write shaders against your types and your conventions. Three things break them when they change:

- **The order of the components.** Index 0 is the first component. If `X` stops being 0, every shader that builds a vector changes meaning.
- **The memory layout your engine uses to upload data.** The attributes never change it, so keep following the rules of the graphics API.
- **Resource names and slots.** Renaming `Frame` or moving it to another set breaks every shader and every reflection test.

Treat a change to any of the three as a major version.

12.3 Shader libraries

A shader library is a normal .NET library. Reuse your types, build on libraries written by others and share yours as a package. Because the types are yours, a library can describe its inputs with the types your engine already defines, and its CPU API comes with it.

12.4 Licenses

Component	License	Where it matters
Yotsuba Shader	MS-PL	Your package can be commercial or not, with no fee
Roslyn	MIT	Inside the compiler
SPIRV-Cross	Apache-2.0	If you use `Transpile` for HLSL or MSL
Naga	MIT or Apache-2.0	If you use `Transpile` for WGSL or GLSL
Khronos SPIR-V grammar	MIT	In the compiler's tables

Keep the notices of the third-party components in the `NOTICE` file of your package. The name and the logo of Yotsuba Shader are not covered by the license.

12.5 Document it for your users

Your users should not need this guide. Give them three pages: how to add the package, a shader that compiles and draws, and a list of your conventions (sets, units, coordinate system). Use your own types in every example.

CHAPTER 13

Troubleshooting

What you see	Why	What to do
`InconsistentVectorComponentTypeException`	A vector has components of different types	Give all components of the vector one type: `float`, `int` or `uint`
`VectorComponentTypeNotValidException`	A vector component is not `float`, `int` or `uint`	Use one of the three, for example `float` instead of `double`
`InvalidOperationException` on a resource	A field is both `[Uniform]` and `[StorageBuffer]`	Choose one
A struct is treated as a plain group of fields	The type has no `[Vector]`, `[Matrix]`, `[Texture]` or `[Sampler]`	Add the attribute to the type you own, not to a type of another package
The shader class is not found	No method has `[ShaderStage]`, or the class is in another project	Mark at least one method. Only types of the project that references the generator are inspected
`CompiledCode` does not exist	The class is not `partial`, or the generator is not referenced	Make the class `partial` and reference the generator
`Binaries.Msl` is not available	The format is not listed in `YotsubaTargets`	Add `msl` and build again
`The shader has no parameter named …`	The engine cannot provide a resource that the shader has	Add the parameter in the backend, or rename the property
A black screen with no errors	A uniform struct has a different layout on the CPU and on the GPU	Compute the offsets with the helper of Chapter 8 and test them
Two attributes with the same name	Two syntax namespaces imported in one file	Import one syntax namespace per file

Appendix A: Checklist

Use it before you publish a new version of your engine.

	Item
☐	Every public vector, matrix, texture and sampler type has its attribute
☐	One shader that uses each of those types compiles without errors
☐	`YotsubaTargets` lists every graphics API the engine supports, and no other
☐	The binding convention is documented and used through constants
☐	`ShaderBase` validates the reflection when a shader loads
☐	Every uniform struct has a layout test
☐	Every shader has a reflection test with the names the engine expects
☐	CI builds, runs the tests and validates the SPIR-V
☐	The package carries the notices of its third-party components
☐	A change to component order, memory layout, resource names or slots bumps the major version

Appendix B: Built-in values across syntaxes

The same value has a different name in each vocabulary. All of them mean the same thing to the compiler.

Concept	HLSL	GLSL	MSL	SPIR-V	WGSL
Vertex stage					
Clip-space position (vertex output)	`SV_Position`	`gl_Position`	`position`	`BuiltIn_Position`	`@builtin_position`
Vertex index	`SV_VertexID`	`gl_VertexIndex`	`vertex_id`	`BuiltIn_VertexIndex`	`@builtin_vertex_index`
Instance index	`SV_InstanceID`	`gl_InstanceIndex`	`instance_id`	`BuiltIn_InstanceIndex`	`@builtin_instance_index`
Base vertex	`SV_StartVertexLocation`	`gl_BaseVertex`	`base_vertex`	`BuiltIn_BaseVertex`	—
Base instance	`SV_StartInstanceLocation`	`gl_BaseInstance`	`base_instance`	`BuiltIn_BaseInstance`	—
Clip distance	`SV_ClipDistanceN`	`gl_ClipDistance`	`clip_distance`	`BuiltIn_ClipDistance`	`@builtin_clip_distances`
Render target layer	`SV_RenderTargetArrayIndex`	`gl_Layer`	`render_target_array_index`	`BuiltIn_Layer`	—
Viewport index	`SV_ViewportArrayIndex`	`gl_ViewportIndex`	`viewport_array_index`	`BuiltIn_ViewportIndex`	—
Fragment stage					
Fragment coordinate	`SV_Position`	`gl_FragCoord`	`position`	`BuiltIn_FragCoord`	`@builtin_position`
Front-facing flag	`SV_IsFrontFace`	`gl_FrontFacing`	`front_facing`	`BuiltIn_FrontFacing`	`@builtin_front_facing`
Fragment depth (output)	`SV_Depth`	`gl_FragDepth`	`depth`	`BuiltIn_FragDepth`	`@builtin_frag_depth`
Conservative depth	`SV_DepthGreaterEqual` / `SV_DepthLessEqual`	—	`depth("greater")` / `depth("less")`	—	—
Stencil reference (output)	`SV_StencilRef`	`gl_FragStencilRefARB`	`stencil`	`BuiltIn_FragStencilRef`	—
Sample index	`SV_SampleIndex`	`gl_SampleID`	`sample_id`	`BuiltIn_SampleId`	`@builtin_sample_index`
Sample mask	`SV_Coverage`	`gl_SampleMask` / `gl_SampleMaskIn`	`sample_mask`	`BuiltIn_SampleMask`	`@builtin_sample_mask`
Primitive index	`SV_PrimitiveID`	`gl_PrimitiveID`	`primitive_id`	`BuiltIn_PrimitiveId`	`@builtin_primitive_index`
Barycentric coordinates	`SV_Barycentrics`	`gl_BaryCoordEXT`	`barycentric_coord`	`BuiltIn_BaryCoord`	—
Barycentric coordinates (no perspective)	—	`gl_BaryCoordNoPerspEXT`	—	`BuiltIn_BaryCoordNoPersp`	—
Helper invocation	—	`gl_HelperInvocation`	—	`BuiltIn_HelperInvocation`	—
Sample position	—	`gl_SamplePosition`	—	`BuiltIn_SamplePosition`	—
Compute stage					
Global invocation id	`SV_DispatchThreadID`	`gl_GlobalInvocationID`	`thread_position_in_grid`	`BuiltIn_GlobalInvocationId`	`@builtin_global_invocation_id`
Local invocation id	`SV_GroupThreadID`	`gl_LocalInvocationID`	`thread_position_in_threadgroup`	`BuiltIn_LocalInvocationId`	`@builtin_local_invocation_id`
Local invocation index	`SV_GroupIndex`	`gl_LocalInvocationIndex`	`thread_index_in_threadgroup`	`BuiltIn_LocalInvocationIndex`	`@builtin_local_invocation_index`

Concept	HLSL	GLSL	MSL	SPIR-V	WGSL
Workgroup id	`SV_GroupID`	`gl_WorkGroupID`	`threadgroup_position_in_grid`	`BuiltIn_WorkgroupId`	`@builtin_workgroup_id`
Number of workgroups	—	`gl_NumWorkGroups`	`threadgroups_per_grid`	`BuiltIn_NumWorkgroups`	`@builtin_num_workgroups`
Workgroup size (method attribute)	`numthreads`	`layout_local_size`	—	—	`@workgroup_size`
Workgroup size (built-in value)	—	`gl_WorkGroupSize`	`threads_per_threadgroup`	`BuiltIn_WorkgroupSize`	—
Subgroup size	—	`gl_SubgroupSize`	`threads_per_simdgroup`	`BuiltIn_SubgroupSize`	`@builtin_subgroup_size`
Subgroup invocation id	—	`gl_SubgroupInvocationID`	`thread_index_in_simdgroup`	`BuiltIn_SubgroupInvocationId`	`@builtin_subgroup_invocation_id`
Subgroup id	—	`gl_SubgroupID`	`simdgroup_index_in_threadgroup`	`BuiltIn_SubgroupId`	`@builtin_subgroup_id`
Number of subgroups	—	`gl_NumSubgroups`	`simdgroups_per_threadgroup`	`BuiltIn_NumSubgroups`	`@builtin_num_subgroups`

Appendix C: Resource attributes across syntaxes

Concept	HLSL	GLSL	MSL	SPIR-V	WGSL
Uniform buffer	`cbuffer, ConstantBuffer<T>`	`uniform`	`constant`	`Uniform`	`uniform`
Read/write storage buffer	`RWStructuredBuffer<T>`	`buffer`	`device`	`StorageBuffer`	`storage(storage_use.read_write)`
Read-only storage buffer	`StructuredBuffer<T>`	`buffer`	`device(readOnly: true)`	`StorageBuffer`	`storage(storage_use.read)`
Binding slot	`register(RegisterPrefix.b, n)`	`layout(set, binding)`	`buffer(n), texture(n), sampler(n)`	`DescriptorSet(s) + Binding(n)`	`@group(s) + @binding(n)`
Texture slot	`register(RegisterPrefix.t, n)`	`layout(set, binding)`	`texture(n)`	`DescriptorSet(s) + Binding(n)`	`@group(s) + @binding(n)`
Sampler slot	`register(RegisterPrefix.s, n)`	`layout(set, binding)`	`sampler(n)`	`DescriptorSet(s) + Binding(n)`	`@group(s) + @binding(n)`

Appendix D: Glossary

Term	Meaning
Entry point	A method marked with `[ShaderStage]`. The GPU starts running a stage there
Stage	A step of the pipeline: vertex, fragment or compute
Location	A number that links an output of one stage to an input of the next
Built-in	A value that the pipeline provides or consumes, such as the clip-space position
Uniform	A value that stays the same during a draw or dispatch
Storage buffer	An array that a shader can read and write
Set and binding	Two numbers that say where a resource is bound
Reflection	The description of a shader: its formats, entry points and resources
SPIR-V	The binary shader format that the compiler writes. The other formats are translated from it
SPIRV-Cross, Naga	The translators that write HLSL and MSL, and WGSL and GLSL
`std140`	A set of rules for how the fields of a struct sit in a uniform buffer
`[ShaderIgnore]`	An attribute that tells Yotsuba Shader that a member is CPU code, not part of the shader

Appendix E: Frequently asked questions

Question	Answer
Does my game need Roslyn or the compiler at run time?	No. The compiler runs during the build. Only the generated binaries, which are static data in your assembly, reach the game.
Can I ship only SPIR-V and translate at run time?	Yes, with the `Transpile` API. It is meant for editors and tools. For a shipped game, translate at build time and avoid carrying a translator in the player.
Which formats do I need for a Direct3D engine?	Only `hlsl`. Compile the text with the DXC or FXC you already use. Yotsuba Shader does not produce DXIL or DXBC.
Can two shaders use different syntaxes?	Yes. The syntax does not change the result. Import one syntax namespace per file.
Do I have to make my shader classes `partial`?	Only when you use the source generator, because it adds the other half of the class. The command line does not need it.
What does `[ShaderIgnore]` apply to?	To any member or type that is CPU code: fields, properties, methods and whole classes, such as a base class.
Can my base class and the generated half coexist?	Yes. The generated half adds no base class, so any class you inherit from works, including the type of another engine.
What if my graphics API is not on the list?	Choose the closest format from `YotsubaTargets`. SPIR-V is always available, and the text formats cover most APIs.
Can I read SPIR-V back into C#?	That direction is planned and has no API yet.